

Временное сохранение изменений (Stashing) в Git

Что такое Git Stash?

Git Stash — это мощный инструмент, который позволяет временно сохранить изменения, над которыми вы работаете, чтобы вы могли переключиться на другую задачу, а затем вернуться к своей работе позже. Это особенно полезно, когда вам нужно быстро переключиться на другую ветку, но вы не хотите создавать коммит с незавершенной работой.

Основные команды Stash

Сохранение изменений

```
# Базовое сохранение всех отслеживаемых изменений
git stash

# Сохранение с сообщением
git stash save "Описание сохраняемых изменений"

# Сохранение включая неотслеживаемые файлы
git stash -u
git stash save -u "Сохранение с неотслеживаемыми файлами"

# Сохранение включая игнорируемые и неотслеживаемые файлы
git stash -a
git stash save -a "Сохранение всех файлов"
```

Просмотр сохранений

```
# Просмотр списка сохранений
git stash list

# Просмотр содержимого последнего сохранения
git stash show

# Просмотр содержимого конкретного сохранения
git stash show stash@{1}

# Просмотр полного diff для сохранения
git stash show -p
git stash show -p stash@{1}
```

Применение сохранений

```
# Применение последнего сохранения (без удаления из списка)
git stash apply

# Применение конкретного сохранения
git stash apply stash@{2}

# Применение последнего сохранения и удаление его из списка
git stash pop

# Применение конкретного сохранения и удаление его из списка
git stash pop stash@{2}
```

Удаление сохранений

```
# Удаление последнего сохранения
git stash drop

# Удаление конкретного сохранения
git stash drop stash@{1}

# Удаление всех сохранений
git stash clear
```

Продвинутые техники использования Stash

Создание ветки из сохранения

Вы можете создать новую ветку на основе состояния, в котором было сохранение:

```
# Создание новой ветки из последнего сохранения
git stash branch new-branch

# Создание новой ветки из конкретного сохранения
git stash branch new-branch stash@{1}
```

Частичное сохранение

Вы можете сохранить только часть изменений в файлах:

```
# Интерактивное сохранение
git stash -p
# или
git stash save -p "Частичное сохранение"
```

При выполнении этой команды Git предложит вам выбрать, какие изменения сохранить, а какие оставить в рабочей директории.

Сохранение только определенных файлов

```
# Добавление нужных файлов в индекс
git add file1.txt file2.txt

# Сохранение только проиндексированных файлов
git stash --staged
```

Применение только определенных файлов из сохранения

```
# Проверка содержимого сохранения
git stash show -p stash@{0}

# Извлечение конкретного файла из сохранения
git checkout stash@{0} -- path/to/file.txt
```

Практические сценарии использования Stash

Сценарий 1: Быстрое переключение задач

```
# Вы работаете над функцией A
# Появляется срочная задача B

# Сохраняем текущую работу
git stash save "Незавершенная функция A"

# Переключаемся на ветку для срочной задачи
git checkout -b urgent-fix

# Выполняем срочную задачу
# ...
git commit -m "Исправлена срочная проблема B"

# Возвращаемся к основной ветке
git checkout main

# Возвращаем сохраненные изменения
git stash pop
```

Сценарий 2: Чистая рабочая директория для pull

```
# У вас есть локальные изменения, но вам нужно получить обновления
git stash
git pull
git stash pop
```

Сценарий 3: Перенос изменений между ветками

```
# Вы сделали изменения в неправильной ветке
git stash
git checkout correct-branch
git stash pop
```

Лучшие практики использования Stash

1. Используйте описательные сообщения

```
git stash save "Работа над авторизацией пользователей"
```

2. Регулярно очищайте список сохранений

```
git stash list # проверьте список
git stash drop stash@{3} # удалите ненужные сохранения
```

3. Используйте ветки для долгосрочных изменений

- Stash лучше использовать для краткосрочного хранения
- Для долгосрочных изменений создавайте отдельные ветки

4. Проверяйте содержимое перед применением

```
git stash show -p stash@{0}
```

5. Будьте осторожны с `git stash pop`

- При возникновении конфликтов сохранение не будет удалено из списка
- Используйте `git stash drop` после разрешения конфликтов

Возможные проблемы и их решения

Конфликты при применении сохранения

Если при выполнении `git stash apply` или `git stash pop` возникают конфликты:

1. Разрешите конфликты вручную
2. Добавьте разрешенные файлы в индекс: `git add <resolved-files>`
3. Если вы использовали `git stash pop`, удалите сохранение вручную: `git stash drop`

Потеря сохранений

Если вы случайно удалили сохранение, его можно восстановить, если вы знаете его хеш:

```
# Найдите хеш удаленного сохранения
git fsck --no-reflog | grep commit | cut -d' ' -f3 | xargs git show | grep -B 5
"WIP on"

# Восстановите сохранение по хешу
git stash apply <hash>
```

Заключение

Git Stash — это мощный инструмент для временного сохранения изменений. Он особенно полезен при необходимости быстрого переключения между задачами без создания лишних коммитов.

Правильное использование stash может значительно повысить вашу продуктивность при работе с Git.

Полезные ресурсы

- [Официальная документация Git Stash](#)
- [Pro Git Book - Git Tools - Stashing and Cleaning](#)
- [Atlassian Git Tutorial - Git Stash](#)